CLOUD SECURITY

UNIT 3 ASSIGNMENT

Set Up a Web Application on Amazon EC2

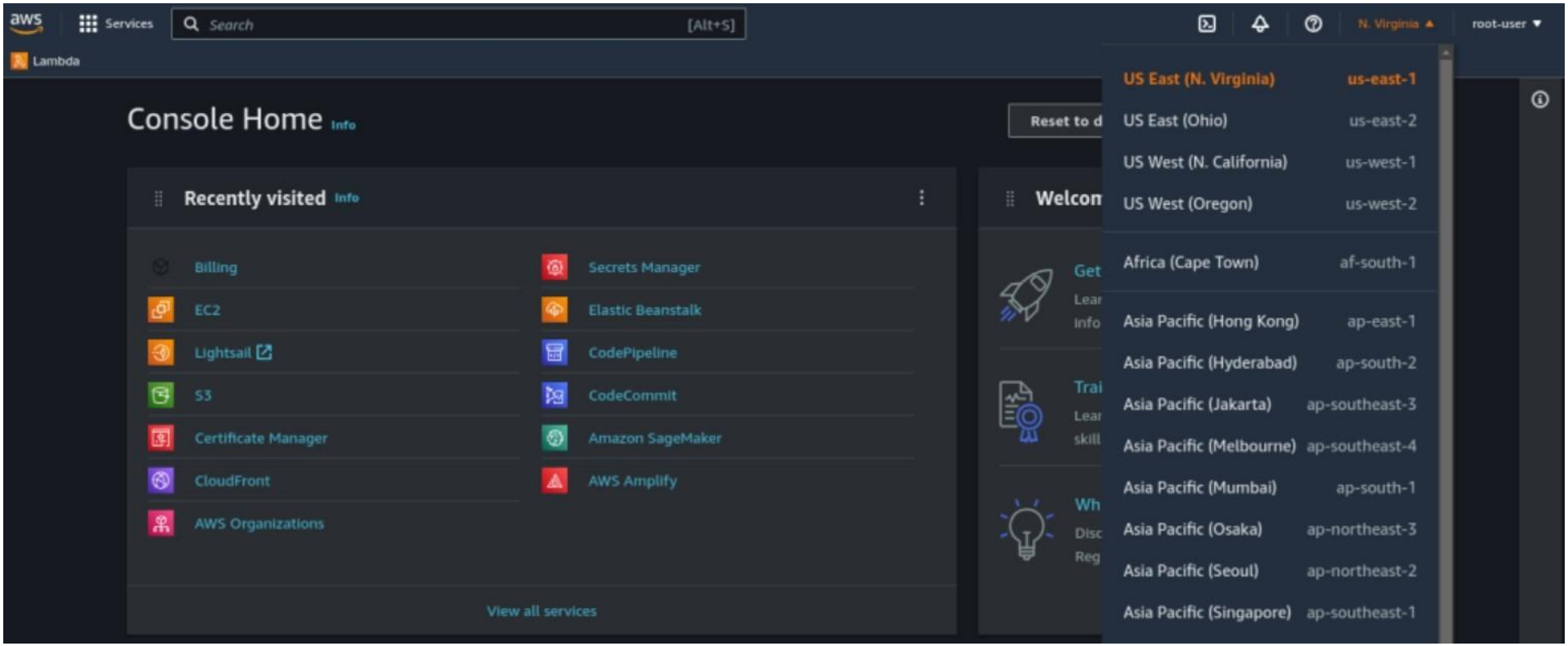
Step-by-step practical report with configuration, commands, expected output, verification, and security considerations

Student Name	Ashish Datta Chavanke
Roll Number	2124UCSM1017
College / University	Sanjivani University
Programme	Cyber Security
Semester	V
Subject	Cloud Security
Submission date	28 October 2026

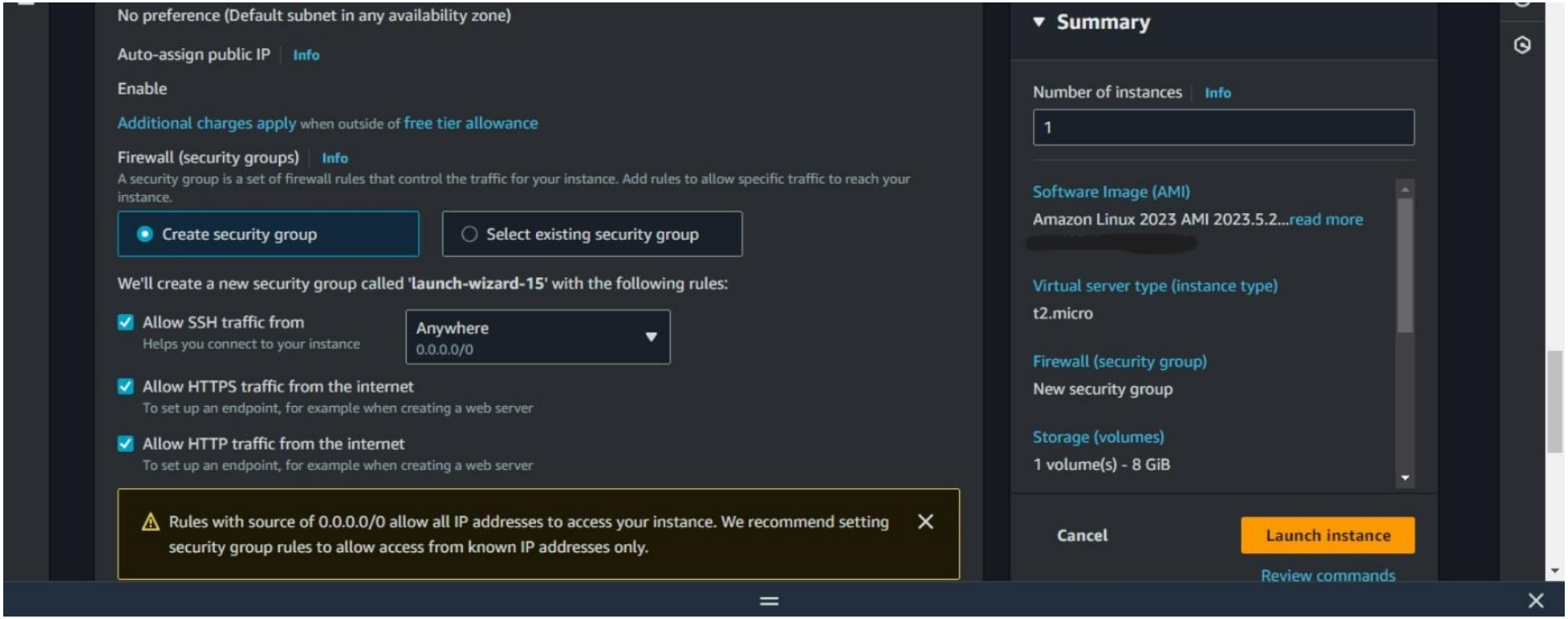
Execution and output note
This report contains commands and representative expected outputs for documentation. The output examples are illustrative and may differ by operating-system image, package version, region, or account settings. They are not represented as captured execution evidence.

AWS Console region selection

Screenshot evidence

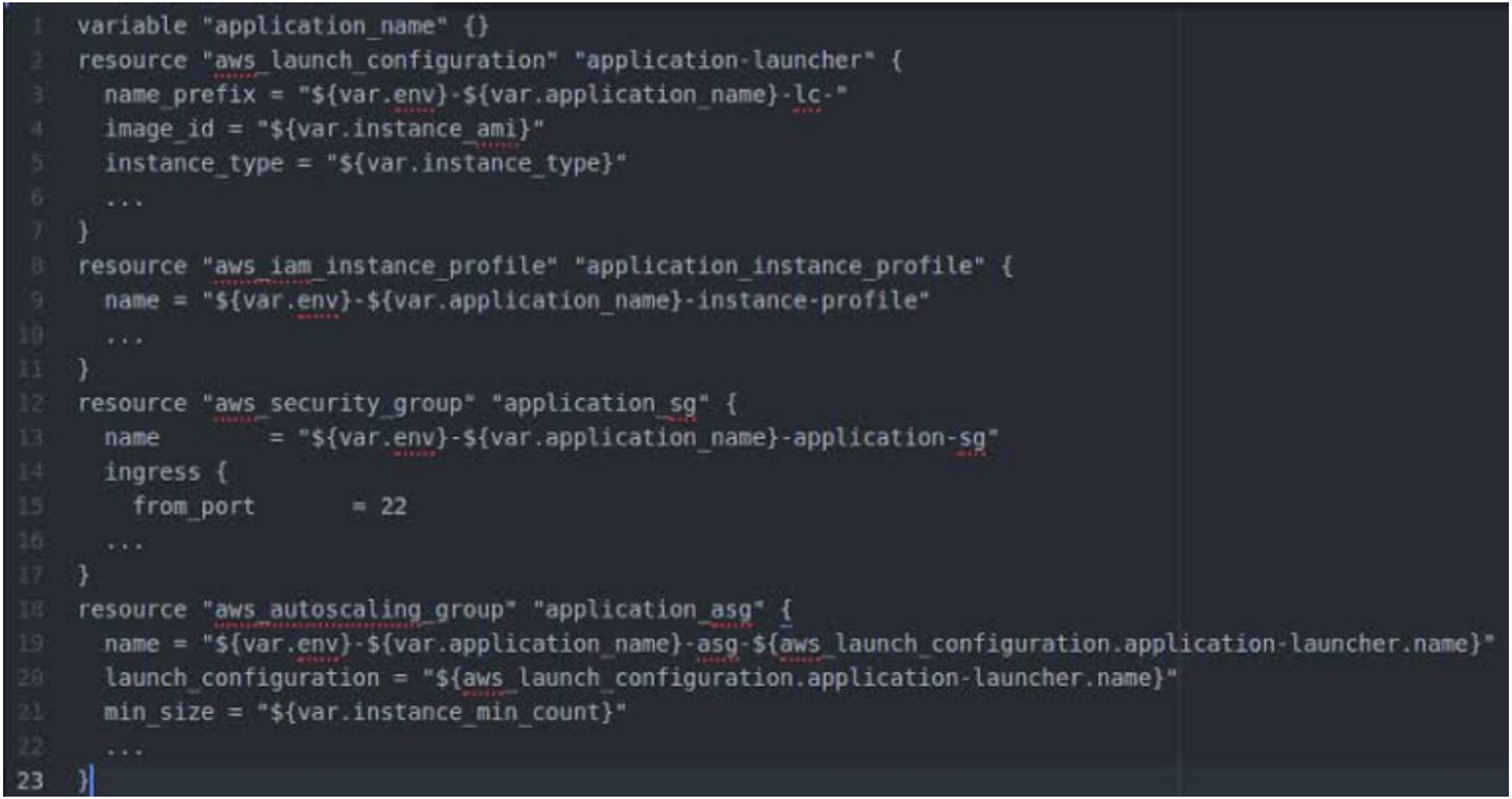


EC2 launch instance configuration



Additional Terraform infrastructure configuration reference

Screenshot evidence



```
variable "application_name" {}
resource "aws_launch_configuration" "application-launcher" {
  name_prefix = "${var.env}-${var.application_name}-lc-"
  image_id = "${var.instance_ami}"
  instance_type = "${var.instance_type}"
  ...
}
resource "aws_iam_instance_profile" "application_instance_profile" {
  name = "${var.env}-${var.application_name}-instance-profile"
  ...
}
resource "aws_security_group" "application_sg" {
  name        = "${var.env}-${var.application_name}-application-sg"
  ingress {
    from_port       = 22
  ...
}
resource "aws_autoscaling_group" "application_asg" {
  name = "${var.env}-${var.application_name}-asg-${aws_launch_configuration.application-launcher.name}"
  launch_configuration = "${aws_launch_configuration.application-launcher.name}"
  min_size = "${var.instance_min_count}"
  ...
}
```

1. Aim

To launch an Amazon EC2 Linux instance, configure restricted SSH and HTTP access, install and start the Apache web server, deploy a simple HTML page, and verify that it is served over HTTP.

2. Requirements

- An AWS account with permission to create EC2 instances, key pairs, and security groups.
- An Amazon Linux 2023 EC2 instance in a suitable AWS Region.
- An SSH client (Linux/macOS terminal, Windows Terminal, or another SSH client).
- A local private key file (.pem) associated with the instance; keep it private.

3. Security Group Configuration

Inbound rule	Protocol / port
SSH	TCP 22 — source: My IP (your current public IPv4 address /32)
HTTP	TCP 80 — source: 0.0.0.0/0 for a public demonstration website; restrict further if required
Outbound	Keep the default outbound rules unless the lab requires a stricter policy

Security note: Never set SSH source to 0.0.0.0/0 for a routine student lab. HTTP is public in this demonstration because the webpage must be reachable from a browser.

4. Procedure

Step 1 — Launch the EC2 instance

- Open AWS Management Console → EC2 → Instances → Launch instances.
- Choose Amazon Linux 2023, select an eligible instance type (for example, a small free-tier-eligible type if available to your account), and select or create an SSH key pair.
- Create or select a security group using the inbound rules above. Enable a public IPv4 address if you need to access the website from the internet.
- Launch the instance and wait until the instance state is Running and status checks pass.

Step 2 — Connect using SSH

Replace the placeholders with your real key path and EC2 public DNS name or IP. On Linux/macOS, set restrictive key-file permissions before connecting.

```
chmod 400 my-key.pem
ssh -i my-key.pem ec2-user@<EC2-PUBLIC-DNS-OR-IP>
```

Representative connection output (exact wording varies):

```
The authenticity of host 'ec2-...compute.amazonaws.com' can't be established.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
[ec2-user@ip-10-0-1-25 ~]$
```

Step 3 — Install Apache and start the service

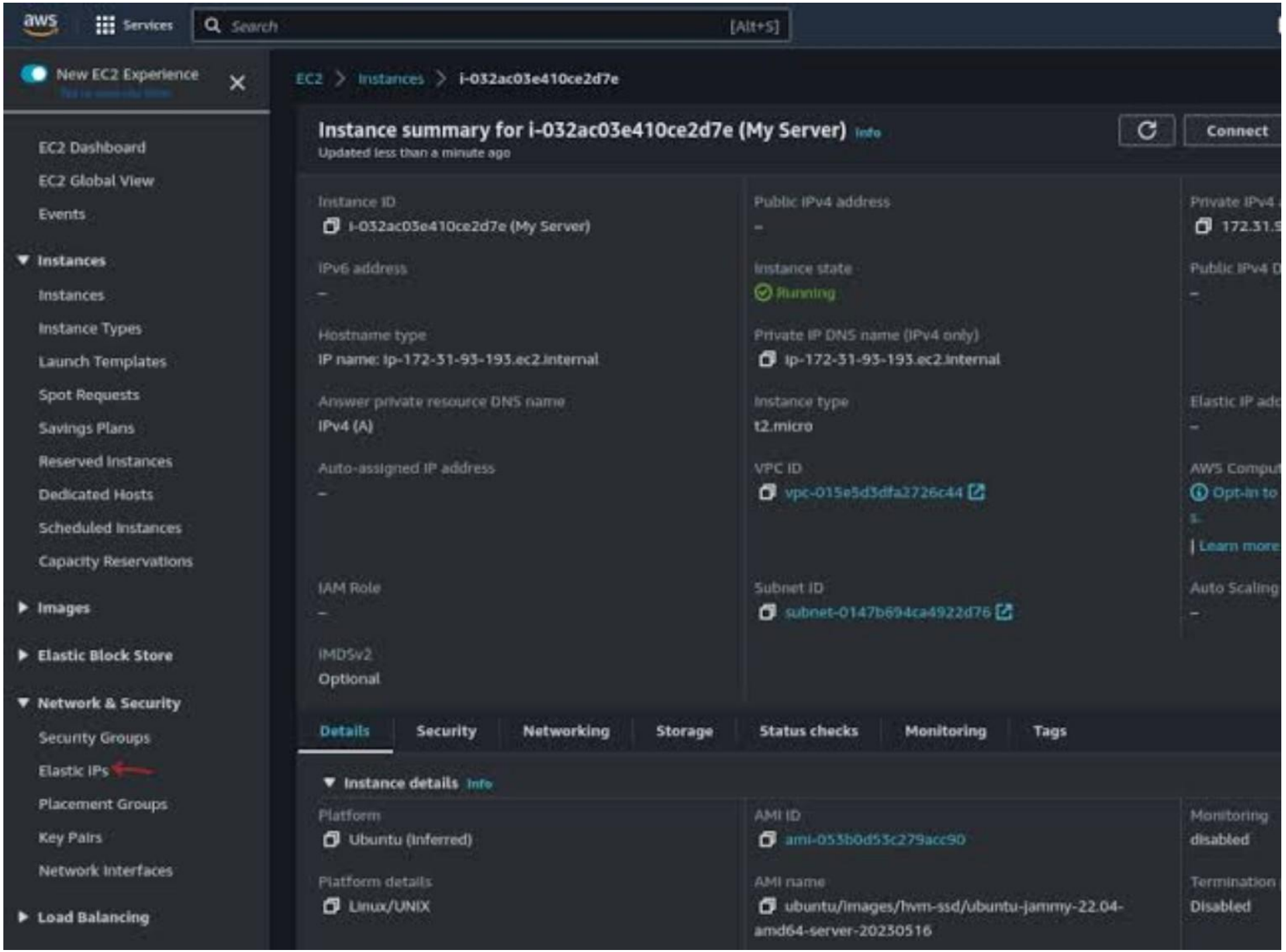
```
sudo dnf update -y
sudo dnf install -y httpd
sudo systemctl enable --now httpd
sudo systemctl status httpd --no-pager
```

Representative status output:

```
● httpd.service - The Apache HTTP Server
     Loaded: loaded (.../httpd.service; enabled)
```

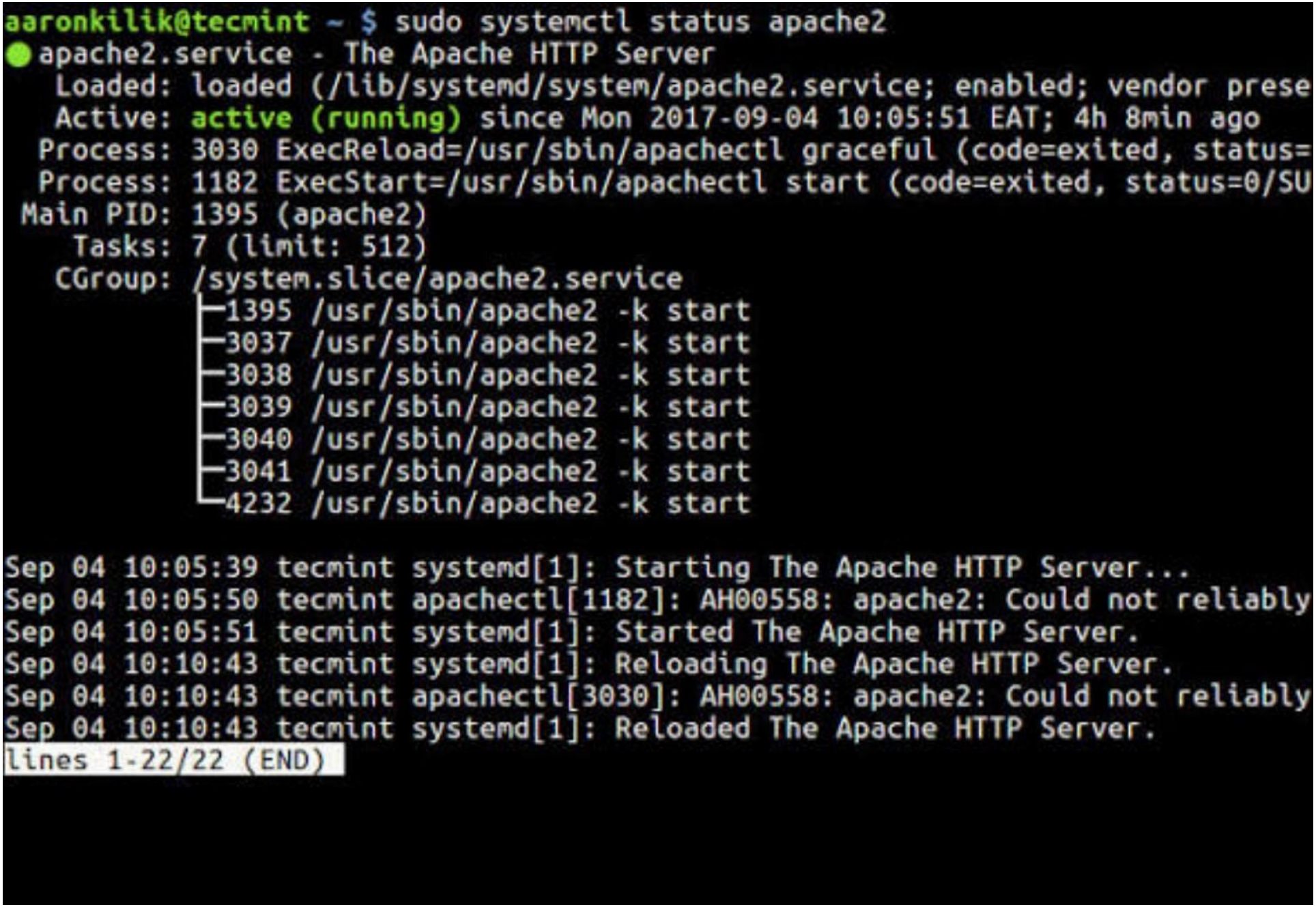
EC2 instance summary and instance state

Screenshot evidence



Apache HTTP Server service status (apache2)

Screenshot evidence



```
     Active: active (running)
```

Step 4 — Create a simple HTML web page

```
sudo tee /var/www/html/index.html > /dev/null <<'EOF'
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Cloud Security Lab</title>
</head>
<body>
  <h1>Welcome to my EC2 Web Application</h1>
  <p>Unit 3 - Cloud Security Assignment</p>
  <p>Hosted using Apache on Amazon EC2.</p>
</body>
</html>
EOF
sudo chown root:root /var/www/html/index.html
sudo chmod 644 /var/www/html/index.html
```

Step 5 — Verify the webpage from the instance

```
curl -I http://localhost
curl http://localhost
```

Representative output:

```
HTTP/1.1 200 OK
Date: <server-generated date>
Server: Apache
Content-Type: text/html; charset=UTF-8

<h1>Welcome to my EC2 Web Application</h1>
<p>Unit 3 - Cloud Security Assignment</p>
<p>Hosted using Apache on Amazon EC2.</p>
```

Step 6 — Verify in a browser

From a device that can reach the instance, open **http://<EC2-PUBLIC-IP>**. The browser should display the heading “Welcome to my EC2 Web Application”. If it does not, verify the instance has a public IP, the HTTP rule permits your test source, and Apache is active.

5. Expected Results and Validation

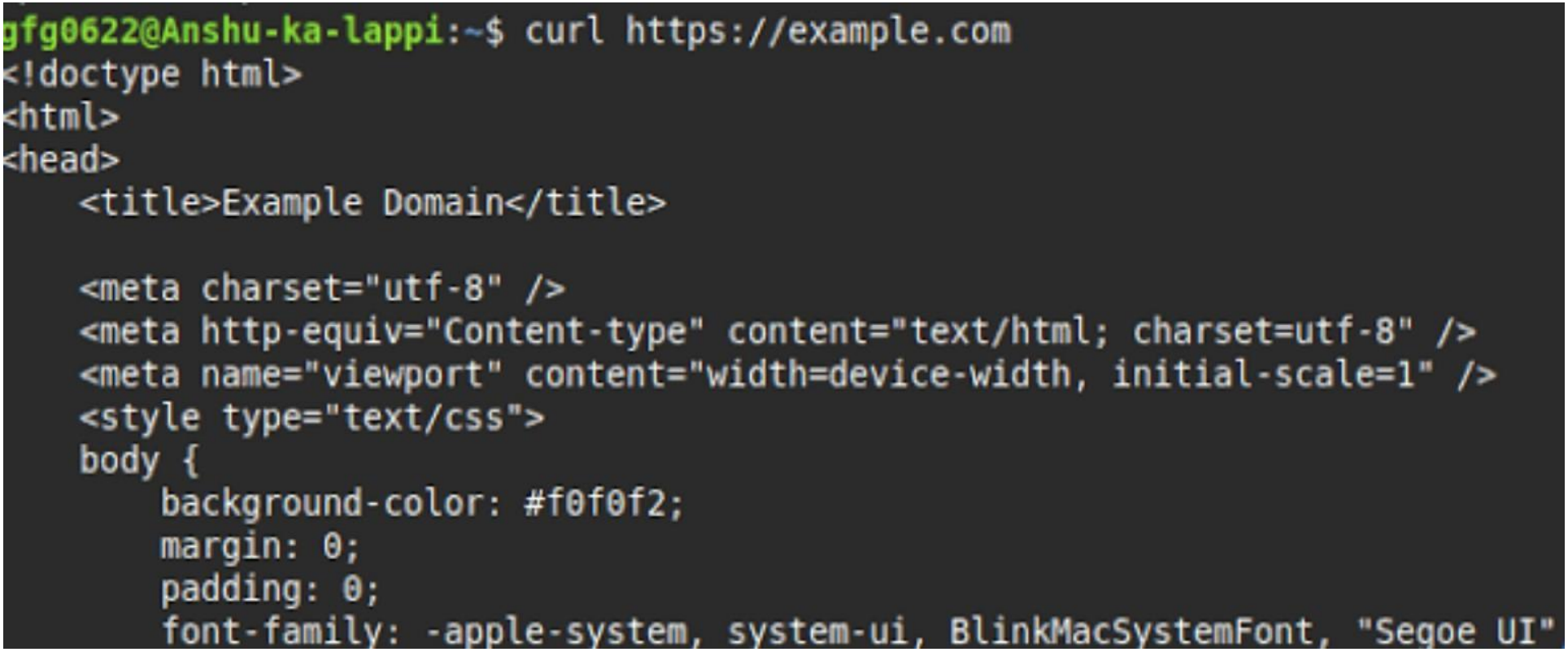
- The EC2 instance is in the Running state and its status checks pass.
- SSH access succeeds using the correct private key and restricted source IP.
- The Apache service is enabled and active.
- curl returns HTTP 200 OK and displays the HTML content.
- The public browser request displays the deployed webpage when the network and security rules permit it.

6. Troubleshooting

SSH timeout	Check instance state, public IP, route table/internet gateway, and TCP 22 source IP rule.
Permission denied (publickey)	Check the login user (ec2-user for Amazon Linux), key pair, and private-key path/permissions.
Website not loading	Check TCP 80 inbound rule, Apache status, instance public IPv4, and subnet routing.
HTTP 403/404	Check the document root and file path; review Apache logs with sudo journalctl -u httpd.

curl command output showing HTML response

Screenshot evidence



7. Clean-up

After evaluation, stop or terminate the instance as instructed by the faculty. Terminating deletes the instance and can remove associated ephemeral storage; review attached volumes and other billable resources. Do not delete a resource needed for grading before confirming submission requirements.

8. Conclusion

This practical demonstrates how to host a basic HTML application on an EC2 Linux instance using Apache. It also demonstrates the importance of security-group rules, key-based SSH authentication, service verification, and testing the website through both curl and a web browser.

9. References

- Amazon EC2 User Guide: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html
- Connect to a Linux instance using SSH: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect-linux-inst-ssh.html
- Security group rules: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/security-group-rules.html
- Amazon Linux 2023 User Guide: https://docs.aws.amazon.com/linux/al2023/ug/
- Apache HTTP Server documentation: https://httpd.apache.org/docs/